COURSE 2A

C2-02 ADVANCED SQL

BEGINNER-FIRST POSTGRESQL TEACHING BOOK - RC1

Senior / BI Analyst optional depth path

Grain-safe analytical SQL -> validation -> performance awareness -> handoff

THIS BOOK IS YOUR TEACHER

PostgreSQL 18 controls syntax. Preserved MySQL video segments are visual support only.

This book is your teacher

How to learn advanced SQL without memorizing finished queries

THE STUDY LOOP

For every lesson: understand the business question -> state input/output grain -> learn vocabulary -> follow one query -> predict shape before running -> do a fresh task -> validate independently -> explain it back -> save attempt -> protected review -> changed retry.

DIALECT RULE

This module is PostgreSQL-first. PostgreSQL 18 documentation controls syntax/behavior. The preserved Alex The Analyst video uses MySQL; use it for visual concepts and chapters only. If syntax differs, follow the book and PostgreSQL reference.

WHAT ADVANCED MEANS

Not obscure syntax. Advanced means grain/cardinality discipline, window/time reasoning, auditable KPI definitions, data-quality investigation, stage-by-stage debugging, plan evidence and maintainable handoff.

MASTERY

A query that runs is not mastered. You must predict the result grain, prove row/key/total behavior, identify one plausible failure mode, and reproduce the competency on changed data without reopening the review.

Contents / Index

Every lesson and actual page number in this RC1 book

Prerequisite and tool map

What Course 1/C2-00 knowledge is assumed and what is new

ASSUMED FROM COURSE 1

SELECT, WHERE, ORDER BY, GROUP BY, HAVING, simple aggregates, basic joins and basic SQL debugging. C2-00 adds grain, metric-contract and validation language. If these basics are not independent, repair Course 1 before using C2-02 as an advanced module.

TOOLS

PostgreSQL 18 preferred for the lab. A PostgreSQL client such as pgAdmin/psql/VS Code database extension may be used. Practice SQL files and synthetic datasets will be provided in the next build step. No paid cloud/database service is required.

NEW DEPTH

Join cardinality; subquery shape; CTE architecture; windows/ranks/LAG; KPI SQL; typed time; quality investigation; validation/debugging; EXPLAIN/index awareness; maintainable analytical handoff.

DIRECT DA -> DE POSITIONING

C2-02 is optional Senior/BI depth. Advanced SQL overlaps strongly with C2B, but the C2B course owns the accelerated engineering-transfer version. Skipping C2A must not block Course 1 -> C2B -> C3 when C2B entry requirements are met.

Practice, review and Gate route

Predict -> run -> validate -> explain -> changed retry

LESSON RULE

Before running SQL, write the expected result grain and one validation query. Save the genuine attempt even if wrong. Protected review opens only after the attempt. Close review before changed retry.

MODULE PRACTICE

The forthcoming lab uses synthetic PostgreSQL tables with deliberate duplicates, orphans, time-boundary traps and join fan-out risks. Two Mini-Labs will integrate SQL reasoning before Gate A.

GATES

Gate A is independent. B/C are different protected full reassessments. A critical wrong grain, denominator, join, time boundary or validation failure blocks pass even if the query executes.

ASQL1 - Advanced joins and grain control

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

The most dangerous SQL join errors often run successfully and produce believable totals. If a one-to-many or many-to-many relationship was not expected, revenue or counts can multiply silently.

WHAT YOU LEARN

State row grain and key/cardinality on both sides before joining. Predict output grain, select the join kind deliberately, surface unmatched rows and prove whether any fan-out is expected.

PREREQUISITE

Course 1 SELECT/WHERE/GROUP BY and basic INNER/LEFT JOIN. C2-00 grain language should be familiar.

NEW WORDS BEFORE WE USE THEM

Cardinality: How many matching rows may exist on each side of a relationship. Fan-out: Unexpected row multiplication after a join. Orphan key: A foreign/business key with no matching parent/master row.

ASQL1 - Follow with me once

Page B - Orders and line items without double counting

Orders and line items without double counting

1. Declare orders = one row per order_id. Declare order_items = one row per order_id + line_id.
2. Do not join order-level order_value to item rows and then SUM(order_value), because each order value would repeat once per item.
3. First aggregate line items to the order grain: WITH line_totals AS (SELECT order_id, SUM(quantity*unit_price) AS line_value FROM order_items GROUP BY order_id).
4. Join orders to line_totals on order_id. Predict one output row per order.
5. Validate with COUNT(*), COUNT(DISTINCT order_id), unmatched order_ids, and an independent total. If rows > distinct orders, investigate instead of adding DISTINCT.

EXPECTED RESULT

One row per intended order with no unexplained multiplication and a saved orphan/fan-out check.

WHY IT WORKED

The many-side table was reduced to the required output grain before the join, so the join contract matched the intended result.

ASQL1 - Now do it yourself

Page C - fresh task, mistakes, recovery and validation

SMALL INDEPENDENT TASK

Fresh case: customers, subscriptions and invoice_lines. Produce one row per subscription with billed value. One customer has multiple subscriptions and one invoice references a missing subscription.

COMMON BEGINNER MISTAKES

1. Joining before writing the two row-grain sentences. | 2. Using DISTINCT to hide fan-out. | 3. Using INNER JOIN when unmatched source rows must remain visible. | 4. Validating only the final total without row/key checks.

STUCK? RECOVERY

Run each table alone. Record rows and distinct business keys. Join only keys first. Compare row count before/after. List duplicates and orphans. Repair the first stage where predicted grain breaks. Save the genuine attempt before protected review.

VALIDATE

State input/output grain, show distinct-key parity, list unmatched keys, and reconcile billed value independently.

ASQL1 - Prove mastery

Page D - professional version, teach-back, protected review and evidence

THEN SHOW THE PROFESSIONAL VERSION

Production analytical SQL usually documents expected cardinality and validates it. A reviewer should be able to see why row count changed and whether that change was intended.

TEACH IT BACK

Explain why a join can be syntactically correct and analytically wrong.

PROTECTED REVIEW + CHANGED RETRY

Save the query and validation first. Then review; close it and retry on a different ticket-event/customer case.

EVIDENCE / MASTERY

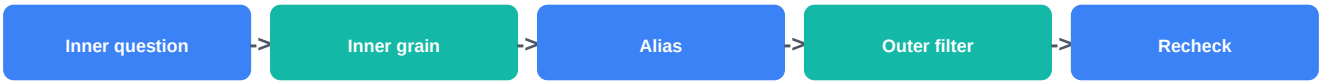
Runnable SQL + grain/cardinality note + unmatched check + row/total reconciliation + explain-back.

VIDEO SUPPORT: Alex The Analyst - 51:10-1:08:03 joins

https://www.postgresql.org/docs/current/tutorial-join.html

ASQL2 - Subqueries and derived tables

Page A - why, outcome, prerequisite, mental model and vocabulary



WHY IT MATTERS

Nested SQL becomes confusing when the inner query has no clear purpose or grain. A useful subquery should simplify one logical stage and make the final question easier to validate.

WHAT YOU LEARN

Use scalar, IN/EXISTS and derived-table subqueries appropriately; identify inner result shape; alias derived tables; avoid accidental correlated work when a simpler staged query will do.

PREREQUISITE

ASQL1 joins and GROUP BY/aggregates.

NEW WORDS BEFORE WE USE THEM

Scalar subquery: A subquery expected to return one value. Derived table: A subquery in FROM treated like a temporary result for that SELECT. Correlated subquery: A subquery that refers to values from the outer query and may be evaluated in relation to each outer row.

ASQL2 - Follow with me once

Page B - Customers above average customer revenue

Customers above average customer revenue

1. Stage 1: create one row per customer with SUM(order_value) after the approved status filter.
2. Inspect that stage alone. Its grain is one row per customer.
3. Stage 2: compute AVG(revenue) from that customer-level result. This must return one scalar value.
4. Outer query filters customer rows where revenue > the scalar average.
5. Validate by saving the scalar average and checking the minimum returned revenue is greater than it.

EXPECTED RESULT

A nested query where every inner result has a stated shape and can be run independently for debugging.

WHY IT WORKED

The average is computed at customer grain instead of order grain, so the business question and denominator align.

ASQL2 - Now do it yourself

Page C - fresh task, mistakes, recovery and validation

SMALL INDEPENDENT TASK

Fresh case: return products whose monthly revenue is above the average monthly product revenue for the same period.

COMMON BEGINNER MISTAKES

1. Computing average order value when the question says average customer revenue. | 2. Writing a scalar subquery that returns multiple rows. | 3. Nesting before testing the inner result. | 4. Using subqueries when named CTE stages would be clearer.

STUCK? RECOVERY

Run the deepest subquery alone. Write its expected columns, row count and grain. Fix it before adding the next level. Save the genuine attempt before protected review.

VALIDATE

Record the inner average, count returned rows and prove every returned value passes the threshold.

ASQL2 - Prove mastery

Page D - professional version, teach-back, protected review and evidence

THEN SHOW THE PROFESSIONAL VERSION

Choose subqueries for compact local logic; choose CTEs when named stages improve review and validation. Maintainability matters more than showing nesting skill.

TEACH IT BACK

Explain a derived table as a temporary result inside one query without calling it a permanent table.

PROTECTED REVIEW + CHANGED RETRY

Attempt first. Changed retry uses EXISTS to find customers with at least one qualifying event.

EVIDENCE / MASTERY

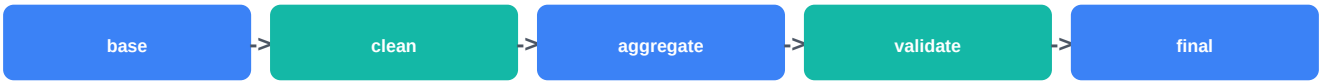
Nested SQL + inner-result screenshots/text + independent threshold check + explain-back.

VIDEO: intentionally book-led for this lesson

https://www.postgresql.org/docs/current/sql-select.html

ASQL3 - CTEs and readable query architecture

Page A - why, outcome, prerequisite, mental model and vocabulary



WHY IT MATTERS

A 70-line query with repeated filters can be correct today and dangerous tomorrow. Named stages let another analyst test each transformation and prevent business rules from drifting across repeated fragments.

WHAT YOU LEARN

Use WITH CTEs to name logical stages, centralize filters, document stage grain and validate intermediate outputs. Understand that CTEs are query expressions, not automatically stored tables.

PREREQUISITE

ASQL2 subqueries and ASQL1 grain control.

NEW WORDS BEFORE WE USE THEM

CTE: Common Table Expression: a named query result defined in a WITH clause for use in the statement. Stage grain: The row meaning produced by one CTE stage. Materialization: Whether PostgreSQL stores/evaluates a CTE result separately versus integrating it into planning; behavior can matter for performance.

ASQL3 - Follow with me once

Page B - Readable customer-month revenue pipeline

Readable customer-month revenue pipeline

1. base: filter source rows once using the approved business rule.
2. customer_month: aggregate base to one row per customer + month.
3. validated: add only checks or classifications needed before final output; do not hide bad rows.
4. final SELECT: choose the business-facing columns and deterministic ORDER BY.
5. Run each CTE by temporarily selecting from it to verify row count, distinct keys and control totals.

EXPECTED RESULT

A query whose stages have meaningful names and whose stage grain can be explained independently.

WHY IT WORKED

Repeated logic became one controlled stage, making drift and debugging easier to detect.

ASQL3 - Now do it yourself

Page C - fresh task, mistakes, recovery and validation

SMALL INDEPENDENT TASK

Fresh case: build eligible_orders -> customer_week -> ranked_customer_week -> final_exception_report.

COMMON BEGINNER MISTAKES

1. Using CTE names like a,b,c that reveal nothing. | 2. Repeating slightly different status filters in multiple CTEs. | 3. Assuming every CTE improves performance. | 4. Leaving validation only in comments without runnable queries.

STUCK? RECOVERY

Collapse the query to base only. Add one CTE at a time. After each addition, run a control SELECT from that CTE. Save the genuine attempt before protected review.

VALIDATE

For every stage save rows, distinct business keys and one control total appropriate to the grain.

ASQL3 - Prove mastery

Page D - professional version, teach-back, protected review and evidence

THEN SHOW THE PROFESSIONAL VERSION

In production SQL, stage names often communicate contracts: source_scope, deduped, enriched, customer_day, validated, final. Keep one responsibility per stage where practical.

TEACH IT BACK

Explain why CTEs improve reviewability even when they do not make the query faster.

PROTECTED REVIEW + CHANGED RETRY

Attempt first. Changed retry restructures a nested query into named stages.

EVIDENCE / MASTERY

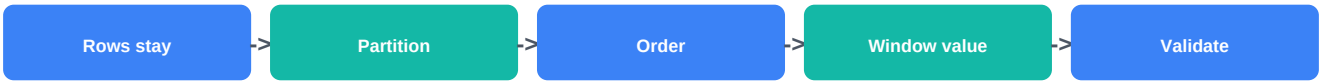
CTE query + stage-grain table + stage controls + explain-back.

VIDEO SUPPORT: Alex The Analyst - 1:58:55-2:09:06 CTEs

https://www.postgresql.org/docs/current/queries-with.html

ASQL4 - Window functions fundamentals

Page A - why, outcome, prerequisite, mental model and vocabulary



WHY IT MATTERS

GROUP BY collapses rows. Many business questions need a group calculation while still keeping each row - for example each order plus the customer's total or department average.

WHAT YOU LEARN

Recognize when a window function is needed; use OVER, PARTITION BY and ORDER BY; explain why row identity is retained; filter window results through a later query stage when necessary.

PREREQUISITE

Aggregates and ASQL3 staged queries.

NEW WORDS BEFORE WE USE THEM

Window function: A calculation across related rows that keeps the individual output rows. PARTITION BY: Divides result rows into groups for the window calculation. OVER: Marks the function as a window calculation and defines its window.

ASQL4 - Follow with me once

Page B - Each order with customer total revenue

Each order with customer total revenue

1. Start with eligible orders at one row per order.
2. Add SUM(order_value) OVER (PARTITION BY customer_id) AS customer_revenue.
3. Compare output row count to input row count; it should remain one row per eligible order.
4. Pick one customer and independently GROUP BY customer_id to confirm the repeated customer_revenue value.
5. If you need top rows based on a window result, calculate the window in a subquery/CTE and filter in the outer SELECT.

EXPECTED RESULT

Window result added without collapsing order rows, with a saved independent grouped check.

WHY IT WORKED

OVER changes the aggregate from row-collapsing behavior to a calculation over a defined set while rows retain identity.

ASQL4 - Now do it yourself

Page C - fresh task, mistakes, recovery and validation

SMALL INDEPENDENT TASK

Fresh case: show every ticket with team average resolution time while keeping one row per ticket.

COMMON BEGINNER MISTAKES

1. Expecting window SUM to reduce rows. | 2. Putting window functions directly in WHERE. | 3. Using PARTITION BY when the business question needs the whole result set. | 4. Forgetting that window ORDER BY affects calculation order, not necessarily final display order.

STUCK? RECOVERY

Remove the window column. Confirm base rows. Add only PARTITION BY first. Validate one partition manually. Add ORDER BY/frame only when the calculation requires it. Save the genuine attempt before protected review.

VALIDATE

Base/output rows equal; one team's window value matches independent GROUP BY result.

ASQL4 - Prove mastery

Page D - professional version, teach-back, protected review and evidence

THEN SHOW THE PROFESSIONAL VERSION

Window functions are excellent for analytical comparisons, but partition, order and frame are part of the business definition and should be reviewed like any other metric logic.

TEACH IT BACK

Explain GROUP BY versus a window function using 'collapse rows' versus 'keep rows'.

PROTECTED REVIEW + CHANGED RETRY

Attempt first. Changed retry uses each transaction plus account-level total.

EVIDENCE / MASTERY

Runnable window SQL + base/output row parity + independent partition check + explain-back.

VIDEO SUPPORT: Alex The Analyst - 1:45:58-1:58:54 window functions

https://www.postgresql.org/docs/current/tutorial-window.html

ASQL5 - Ranking, partitions and running calculations

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Top-N and running totals look simple until ties or duplicate timestamps appear. ROW_NUMBER, RANK and DENSE_RANK answer different questions, and default window frames can surprise you.

WHAT YOU LEARN

Choose ranking function based on tie policy; define deterministic ordering; distinguish partition from frame; write explicit ROWS frames for running calculations when needed.

PREREQUISITE

ASQL4 window basics.

NEW WORDS BEFORE WE USE THEM

ROW_NUMBER: Unique sequential number according to the specified window order. RANK: Ranking with gaps after ties. DENSE_RANK: Ranking without gaps after ties. Frame: The subset of rows within the window used for a particular calculation.

ASQL5 - Follow with me once

Page B - Top orders and running customer spend

Top orders and running customer spend

1. For top order value per customer, decide tie policy before picking ROW_NUMBER/RANK/DENSE_RANK.
2. Add a deterministic tie-breaker such as order_id after order_value if exactly one row must win.
3. For running spend, order by order_date, order_id and write ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW explicitly.
4. Select one customer and manually sort their orders to verify the first three running totals.
5. Document whether two equal order values should share rank or one must be chosen deterministically.

EXPECTED RESULT

Ranking and running-total logic matches a written tie/frame policy.

WHY IT WORKED

The business rule controls ranking and window frame instead of relying on defaults.

ASQL5 - Now do it yourself

Page C - fresh task, mistakes, recovery and validation

SMALL INDEPENDENT TASK

Fresh case: top two products per region by revenue plus cumulative monthly revenue within region.

COMMON BEGINNER MISTAKES

1. Using ROW_NUMBER when ties should share a rank. | 2. Ordering by a non-unique timestamp without tie-breaker. | 3. Assuming default frame always means 'rows so far'. | 4. Using final ORDER BY as if it changed window calculation order.

STUCK? RECOVERY

Create a tiny 5-row example with an intentional tie. Compare ROW_NUMBER/RANK/DENSE_RANK. Then test the frame on the same rows. Save the genuine attempt before protected review.

VALIDATE

Check tie case explicitly and manually recompute one partition's running values.

ASQL5 - Prove mastery

Page D - professional version, teach-back, protected review and evidence

THEN SHOW THE PROFESSIONAL VERSION

Stable analytical outputs use deterministic sort keys. If the decision requires exactly N rows, document how ties are broken.

TEACH IT BACK

Explain why a ranking function choice is a business rule, not only syntax.

PROTECTED REVIEW + CHANGED RETRY

Attempt first. Changed retry uses employee performance scores with deliberate ties.

EVIDENCE / MASTERY

Ranking SQL + tie policy + explicit frame + manual partition check + explain-back.

VIDEO: intentionally book-led for this lesson

https://www.postgresql.org/docs/current/functions-window.html

ASQL6 - LAG, LEAD and time-based comparisons

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Month-over-month or event-to-event comparison is wrong if the rows are not first reduced to the correct time grain and ordered deterministically.

WHAT YOU LEARN

Use LAG/LEAD after defining entity and time grain; handle the first/last row; distinguish missing period from zero; calculate changes only after validating ordering and continuity.

PREREQUISITE

ASQL5 window ordering and ASQL8-style typed dates will reinforce this lesson.

NEW WORDS BEFORE WE USE THEM

LAG: Returns a value from a previous row in the window ordering. LEAD: Returns a value from a following row in the window ordering. Time grain: The time unit represented by one row, such as one row per month.

ASQL6 - Follow with me once

Page B - Month-over-month revenue

Month-over-month revenue

1. First aggregate eligible orders to one row per month using date_trunc('month', order_date).
2. Apply LAG(revenue) OVER (ORDER BY month_start) to the monthly result, not raw order rows.
3. Compute absolute change and, only when denominator policy is defined, percentage change.
4. Check the first month: prior_revenue should be NULL because no earlier row exists.
5. Detect whether a missing calendar month is absent from the data. Missing row is not the same as a zero-revenue month.

EXPECTED RESULT

One row per month with prior value/change and an explicit policy for first/missing periods.

WHY IT WORKED

The time series is normalized to month grain before row-to-row comparison.

ASQL6 - Now do it yourself

Page C - fresh task, mistakes, recovery and validation

SMALL INDEPENDENT TASK

Fresh case: weekly support volume by team with previous-week comparison; one week is missing for one team.

COMMON BEGINNER MISTAKES

1. Using LAG on unsummarized order rows for monthly change. | 2. Treating NULL prior value as zero automatically. | 3. Ignoring missing months. | 4. Ordering by formatted month text instead of typed time.

STUCK? RECOVERY

Run the monthly CTE alone. Confirm one row per period and chronological order. Only then add LAG/LEAD. Save the genuine attempt before protected review.

VALIDATE

Manually check two adjacent periods and explicitly flag the missing-period condition.

ASQL6 - Prove mastery

Page D - professional version, teach-back, protected review and evidence

THEN SHOW THE PROFESSIONAL VERSION

For operational time series, a calendar/date spine may be needed to represent missing periods explicitly. That deeper modeling appears later; here you must at least detect the difference.

TEACH IT BACK

Explain why LAG does not know what 'previous month' means unless your row set and order make it true.

PROTECTED REVIEW + CHANGED RETRY

Attempt first. Changed retry compares next scheduled appointment per patient with LEAD.

EVIDENCE / MASTERY

Time-grain query + LAG/LEAD + missing-period check + manual comparison + explain-back.

VIDEO SUPPORT: Alex The Analyst - 1:45:58-1:58:54 optional window visual; LAG/LEAD taught in-book

https://www.postgresql.org/docs/current/functions-window.html

ASQL7 - Conditional aggregation and advanced KPI queries

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

A KPI query can return a neat percentage while numerator and denominator use different populations. Conditional aggregation must implement the metric contract visibly.

WHAT YOU LEARN

Use CASE and PostgreSQL FILTER for conditional counts/sums; separate numerator and denominator; use NULLIF for defined zero-denominator behavior; reconcile KPI components independently.

PREREQUISITE

C2-00 metric contracts, aggregates, ASQL1 grain.

NEW WORDS BEFORE WE USE THEM

Conditional aggregation: Aggregating only rows that satisfy a condition, often through CASE or FILTER. FILTER clause: PostgreSQL syntax that applies a WHERE-like condition to one aggregate. NULLIF: Returns NULL when two expressions are equal; often used to avoid division by zero after the business rule is defined.

ASQL7 - Follow with me once

Page B - Completion rate and completed revenue by region

Completion rate and completed revenue by region

1. Write the metric contract first: numerator = completed eligible orders; denominator = all eligible non-test orders.
2. Use COUNT(*) FILTER (WHERE status='completed') for numerator when PostgreSQL is the target dialect.
3. Build denominator separately with its own eligible-population filter; do not assume COUNT(*) is correct.
4. Compute rate using numerator::numeric / NULLIF(denominator,0) only after zero-population behavior is agreed.
5. Reconcile numerator and denominator for one region with independent WHERE queries.

EXPECTED RESULT

KPI output whose components can be inspected and independently recomputed.

WHY IT WORKED

Business definitions stay visible in the aggregate expressions instead of being hidden in a final ratio.

ASQL7 - Now do it yourself

Page C - fresh task, mistakes, recovery and validation

SMALL INDEPENDENT TASK

Fresh case: First Contact Resolution Rate by team with an exclusion for reopened-within-48-hours cases.

COMMON BEGINNER MISTAKES

1. Different filters on numerator and denominator by accident. | 2. ELSE NULL versus ELSE 0 confusion without understanding aggregate behavior. | 3. IFERROR-like hiding of zero denominator. | 4. Formatting as percent before validating components.

STUCK? RECOVERY

Remove the rate. Return only numerator and denominator. Validate each separately. Add the division last. Save the genuine attempt before protected review.

VALIDATE

Independent numerator/denominator queries match output; sum of segments reconciles to overall eligible population where appropriate.

ASQL7 - Prove mastery

Page D - professional version, teach-back, protected review and evidence

THEN SHOW THE PROFESSIONAL VERSION

In reviewed SQL, metric components are often named in a CTE so definitions can be tested and reused. Keep business labels close to the exact filters they mean.

TEACH IT BACK

Explain why a percentage should be built last, not first.

PROTECTED REVIEW + CHANGED RETRY

Attempt first. Changed retry builds refund rate with a different denominator.

EVIDENCE / MASTERY

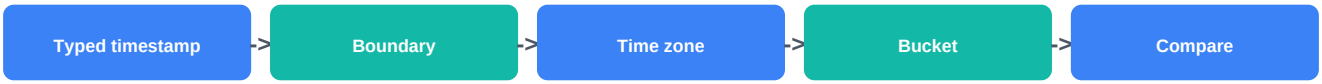
KPI SQL + metric contract + component validation + explain-back.

VIDEO SUPPORT: Alex The Analyst - 1:26:39-1:35:15 CASE statements

https://www.postgresql.org/docs/current/functions-aggregate.html

ASQL8 - Date and time analysis

Page A - why, outcome, prerequisite, mental model and vocabulary



WHY IT MATTERS

Text slicing and inclusive end dates often create hidden period errors. Timestamps also introduce time-zone and boundary questions that must be explicit.

WHAT YOU LEARN

Use typed DATE/TIMESTAMP values, half-open intervals, date_trunc/date_bin where appropriate, and explicit time-zone assumptions. Avoid text-based month logic.

PREREQUISITE

Basic WHERE filters; ASQL6 time-grain reasoning.

NEW WORDS BEFORE WE USE THEM

Half-open interval: A time filter including the start and excluding the next-period boundary, such as >= Apr 1 and < May 1. date_trunc: PostgreSQL function that truncates a date/time value to a requested precision. Time zone: Rules used to interpret local clock time relative to UTC/other zones.

ASQL8 - Follow with me once

Page B - Monthly revenue with safe boundaries

Monthly revenue with safe boundaries

1. Use order_ts >= TIMESTAMP '2026-01-01 00:00:00' AND order_ts < TIMESTAMP '2026-07-01 00:00:00'.
2. Bucket with date_trunc('month', order_ts) after confirming the business reporting time zone.
3. Do not use BETWEEN '2026-01-01' AND '2026-06-30' for timestamp data unless you truly intend the end literal behavior.
4. Check minimum and maximum timestamps returned and count rows exactly at a boundary in a tiny test.
5. If source is timestamptz, record the reporting-zone decision before grouping by local month.

EXPECTED RESULT

Typed, boundary-safe period logic with explicit calendar/time-zone assumptions.

WHY IT WORKED

The next-period exclusive boundary includes every instant in the intended period without guessing the final day's maximum time.

ASQL8 - Now do it yourself

Page C - fresh task, mistakes, recovery and validation

SMALL INDEPENDENT TASK

Fresh case: analyze support events by local business day when timestamps are stored with time zone.

COMMON BEGINNER MISTAKES

1. Comparing dates stored as text. | 2. Using <= end_date with timestamp data carelessly. | 3. Grouping UTC timestamps into local business days without a zone decision. | 4. Formatting first, then grouping on strings.

STUCK? RECOVERY

Inspect column data type and sample minimum/maximum. Test one record at the start boundary and one at the next-period boundary. Save the genuine attempt before protected review.

VALIDATE

Boundary test rows behave as predicted; bucket count reconciles to filtered source count.

ASQL8 - Prove mastery

Page D - professional version, teach-back, protected review and evidence

THEN SHOW THE PROFESSIONAL VERSION

Period logic belongs in metric contracts. Reporting month, operational day and UTC storage can all be different concepts.

TEACH IT BACK

Explain half-open date filtering using a shop open from one boundary until the next.

PROTECTED REVIEW + CHANGED RETRY

Attempt first. Changed retry uses weekly buckets with a different reporting zone.

EVIDENCE / MASTERY

Typed date/time SQL + boundary tests + bucket reconciliation + explain-back.

VIDEO: intentionally book-led for this lesson

https://www.postgresql.org/docs/18/functions-datetime.html

ASQL9 - Data-quality investigation with SQL

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Data quality should be investigated before it contaminates a metric. SQL can turn vague suspicion into exact exception sets that can be counted, owned and repaired.

WHAT YOU LEARN

Write reusable checks for duplicate business keys, null required fields, orphan keys, invalid ranges, unexpected categories and cross-table control totals.

PREREQUISITE

ASQL1 joins/grain and basic aggregates.

NEW WORDS BEFORE WE USE THEM

Profiling query: A query that summarizes shape, values or exceptions to understand data quality. Required field: A field whose missing value makes a business process or analysis invalid. Domain rule: A rule defining valid values/ranges for a field.

ASQL9 - Follow with me once

Page B - Customer and order quality scan

Customer and order quality scan

1. Duplicate customer key: GROUP BY customer_id HAVING COUNT(*) > 1.
2. Missing required field: WHERE region IS NULL OR btrim(region)=''.
3. Orphan order: LEFT JOIN customers and filter c.customer_id IS NULL.
4. Invalid amount: WHERE order_value < 0 when negatives are not allowed by the contract.
5. Unexpected status: compare DISTINCT status against an approved status list; save exception counts rather than silently mapping them.

EXPECTED RESULT

A small exception report where each quality rule has count, sample keys and business impact.

WHY IT WORKED

The checks target different failure classes instead of relying on one generic 'clean data' query.

ASQL9 - Now do it yourself

Page C - fresh task, mistakes, recovery and validation

SMALL INDEPENDENT TASK

Fresh case: investigate ticket, team and customer tables containing duplicate teams, orphan customer IDs and negative resolution hours.

COMMON BEGINNER MISTAKES

1. Removing duplicate rows before identifying which key is duplicated. | 2. Treating all nulls as errors. | 3. Using NOT IN carelessly when null semantics are not understood. | 4. Fixing source values directly without owner approval.

STUCK? RECOVERY

Choose one rule at a time: grain/key, required field, relationship, range, category. Return only exception rows and count them. Save the genuine attempt before protected review.

VALIDATE

Exception counts are reproducible and zero only after an approved repair; raw evidence remains preserved.

ASQL9 - Prove mastery

Page D - professional version, teach-back, protected review and evidence

THEN SHOW THE PROFESSIONAL VERSION

Quality checks become operational when each rule has owner, severity, expected threshold and recovery action. C2-02 focuses on query design; broader observability comes later.

TEACH IT BACK

Explain why data-quality SQL should return the bad rows, not only a green/red flag.

PROTECTED REVIEW + CHANGED RETRY

Attempt first. Changed retry uses subscription/billing quality rules.

EVIDENCE / MASTERY

Quality-check SQL + exception outputs + business-impact note + explain-back.

VIDEO: intentionally book-led for this lesson

https://www.postgresql.org/docs/current/queries-table-expressions.html

ASQL10 - Query debugging and validation

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Changing several clauses at once can make a query appear fixed without proving which error caused the problem. Senior debugging isolates the first broken stage.

WHAT YOU LEARN

Classify syntax versus logic versus data-quality failures; debug stage-by-stage; compare row count/distinct keys/control totals; build validation queries that could catch plausible wrong answers.

PREREQUISITE

ASQL1-ASQL9; especially CTE stages and grain checks.

NEW WORDS BEFORE WE USE THEM

Syntax error: The database cannot parse/execute the statement as written. Logic error: The SQL runs but answers the wrong question. Control total: An independently obtained count/sum used to reconcile the query result.

ASQL10 - Follow with me once

Page B - Find the first stage that doubles revenue

Find the first stage that doubles revenue

1. Record the symptom: final revenue is almost exactly 2x expected.
2. Run base CTE controls: rows, distinct order_id, SUM(order_value).
3. Run the next join stage with the same controls. If rows and total double here, stop; later stages are not the first cause.
4. Inspect right-side join key for duplicates. Do not change filters or add DISTINCT yet.
5. Repair the key/cardinality cause, rerun controls from base through final, and save before/after evidence.

EXPECTED RESULT

A debugging log identifying the first failed stage and proving the repair removed the underlying cause.

WHY IT WORKED

The search space shrinks because every stage has shape and total evidence.

ASQL10 - Now do it yourself

Page C - fresh task, mistakes, recovery and validation

SMALL INDEPENDENT TASK

Fresh case: a monthly customer report loses 12 customers after enrichment. Find the first stage where row count drops and decide whether the loss is expected.

COMMON BEGINNER MISTAKES

1. Editing many clauses before rerunning controls. | 2. Fixing the final SELECT when the error began upstream. | 3. Using the same final query as its own validation. | 4. Deleting problem rows until totals match.

STUCK? RECOVERY

Return to the earliest stable stage. Add one transformation at a time and compare rows, distinct keys and one additive total. Save the genuine attempt before protected review.

VALIDATE

Before/after evidence shows exactly which stage changed shape and why; final controls rerun after repair.

ASQL10 - Prove mastery

Page D - professional version, teach-back, protected review and evidence

THEN SHOW THE PROFESSIONAL VERSION

Keep small diagnostic queries near complex analytical SQL during development. Validation queries are part of the deliverable for important outputs.

TEACH IT BACK

Explain why 'the query runs now' is not proof that the bug is fixed.

PROTECTED REVIEW + CHANGED RETRY

Attempt first. Changed retry uses an incorrect date filter rather than a join bug.

EVIDENCE / MASTERY

Debug log + stage controls + root cause + repair + rerun evidence + explain-back.

VIDEO: intentionally book-led for this lesson

https://www.postgresql.org/docs/current/sql-select.html

ASQL11 - Performance concepts: indexes, scans and EXPLAIN

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Optimization before correctness can make the wrong result faster. Analysts need enough plan awareness to identify obvious scans/row-estimate problems and communicate evidence without pretending to be database administrators.

WHAT YOU LEARN

Read basic EXPLAIN nodes/cost/estimated rows; understand sequential versus index scans conceptually; know that EXPLAIN ANALYZE executes the statement; compare estimated and actual rows; discuss index trade-offs.

PREREQUISITE

Correct, validated SQL from earlier lessons. Basic idea of a database index is taught here.

NEW WORDS BEFORE WE USE THEM

Query plan: The execution strategy PostgreSQL chooses for a statement. Sequential scan: Reading table pages broadly rather than locating rows via an index. Index scan: Using an index to locate candidate rows. Estimate: Planner prediction such as expected rows/cost before actual execution.

ASQL11 - Follow with me once

Page B - Inspect a filtered aggregation

Inspect a filtered aggregation

1. First prove the SELECT is correct with normal validation.
2. Run EXPLAIN SELECT ... to inspect the planned access path without requesting actual execution details.
3. Run EXPLAIN (ANALYZE, BUFFERS) only on a safe SELECT in the practice environment and remember ANALYZE executes it.
4. Compare estimated rows to actual rows and identify the largest mismatch or expensive-looking node; do not optimize from node names alone.
5. Discuss whether an index on the filter/join key could help and what write/storage overhead it adds. Measure before and after if the lab provides an index.

EXPECTED RESULT

A short plan note naming observed scan/join type, estimate-versus-actual evidence and one measured optimization hypothesis.

WHY IT WORKED

Performance reasoning comes after correctness and uses observed plan evidence rather than generic 'indexes make SQL fast' claims.

ASQL11 - Now do it yourself

Page C - fresh task, mistakes, recovery and validation

SMALL INDEPENDENT TASK

Fresh case: compare a selective date/customer filter before and after an approved practice index. Record plan and timing/row evidence.

COMMON BEGINNER MISTAKES

1. Running EXPLAIN ANALYZE on unsafe modifying statements without understanding execution. | 2. Calling every sequential scan bad. | 3. Creating many indexes without workload evidence. | 4. Comparing timing from unrelated environments as if absolute.

STUCK? RECOVERY

Use plain EXPLAIN first. Identify top nodes, estimated rows and predicates. If safe, add ANALYZE in the lab. Compare one change at a time. Save the genuine attempt before protected review.

VALIDATE

Query result stays identical; plan evidence saved; trade-off statement includes index maintenance/storage cost.

ASQL11 - Prove mastery

Page D - professional version, teach-back, protected review and evidence

THEN SHOW THE PROFESSIONAL VERSION

Analysts should know when to escalate a performance issue with reproducible SQL, row counts and EXPLAIN evidence rather than guessing at database configuration.

TEACH IT BACK

Explain why a sequential scan can sometimes be the right plan.

PROTECTED REVIEW + CHANGED RETRY

Attempt first. Changed retry examines a join query with a different selectivity pattern.

EVIDENCE / MASTERY

Validated query + before/after EXPLAIN evidence + result parity + trade-off + explain-back.

VIDEO: intentionally book-led for this lesson

https://www.postgresql.org/docs/current/using-explain.html

ASQL12 - Maintainable production-style analytical SQL

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

A one-off query may answer today. A production-style analytical query must let another analyst understand the decision, assumptions, grain, stages, validation and how to rerun it safely.

WHAT YOU LEARN

Package analytical SQL with header contract, readable CTE stages, consistent names/formatting, deterministic output, separate validations, business interpretation and handoff notes.

PREREQUISITE

All ASQL1-ASQL11.

NEW WORDS BEFORE WE USE THEM

Deterministic output: Output whose ordering/tie behavior is explicitly controlled when order matters. Query contract: Short documentation of decision, scope, grain, assumptions and source tables. Handoff: Information another analyst needs to rerun, validate and modify the analysis safely.

ASQL12 - Follow with me once

Page B - Turn an analyst query into a reviewable deliverable

Turn an analyst query into a reviewable deliverable

1. Header comment: decision, metric definition/version, intended final grain, source tables, period, exclusions and assumptions.
2. WITH stages: base_scope -> quality_checked -> enriched -> business_grain -> final_metrics. Use descriptive aliases.
3. Final SELECT includes only decision-facing columns and deterministic ORDER BY when presentation order matters.
4. Separate validation block checks row/distinct-key counts, orphans and control totals; do not hide it inside comments.
5. Write a 3-5 sentence interpretation: what happened, materiality, uncertainty, recommendation and what would change it.

EXPECTED RESULT

A runnable SQL package another analyst can review stage-by-stage and reconcile without asking what each section means.

WHY IT WORKED

Technical logic, business contract and evidence are packaged together instead of living in separate memory/chat fragments.

ASQL12 - Now do it yourself

Page C - fresh task, mistakes, recovery and validation

SMALL INDEPENDENT TASK

Fresh case: deliver a production-style regional service-performance query with quality exceptions, KPI output and a concise business recommendation.

COMMON BEGINNER MISTAKES

1. Leaving magic dates/status values unexplained. | 2. Using SELECT * in the final output without need. | 3. No deterministic tie/order rule. | 4. Deleting validation queries before handoff. | 5. Writing business conclusions not supported by the query.

STUCK? RECOVERY

Start with the header contract and final grain. Rename each stage for one job. Add one validation per major failure mode. Only then polish formatting. Save the genuine attempt before protected review.

VALIDATE

A second analyst can state source/grain/rules, run validations, reproduce final output and identify one safe modification point.

ASQL12 - Prove mastery

Page D - professional version, teach-back, protected review and evidence

THEN SHOW THE PROFESSIONAL VERSION

Maintainable SQL is boring in a good way: explicit contracts, stable naming, small stages, deterministic logic, validation and evidence. Complexity is justified only when the business problem requires it.

TEACH IT BACK

Explain what makes SQL 'production-style' without using line count or advanced syntax as the definition.

PROTECTED REVIEW + CHANGED RETRY

Attempt first. Changed retry packages a different subscription-health analysis.

EVIDENCE / MASTERY

Final SQL file + validation section + data-quality note + interpretation + handoff + 90-second defense.

VIDEO SUPPORT: Alex The Analyst - 2:42:46-3:32:13 optional cleaning/project workflow

https://www.postgresql.org/docs/current/queries.html

C2-02 mastery checklist + quick reference

Use after the first full pass; not a substitute for writing SQL

TWELVE QUESTIONS

ASQL1 What are both input grains and join cardinality? | ASQL2 What shape does each subquery return? | ASQL3 Can each named stage be validated? | ASQL4 Do rows stay or collapse? | ASQL5 What is the tie/frame rule? | ASQL6 Is the time grain/order complete? | ASQL7 Are numerator/denominator populations aligned? | ASQL8 Are boundaries typed and time-zone aware? | ASQL9 Which exact bad rows violate each rule? | ASQL10 Where is the first broken stage? | ASQL11 What does the plan actually show? | ASQL12 Can another analyst rerun and defend it?

BLOCK MASTERY IF

Wrong/unexplained result grain; hidden fan-out; orphan rows lost unintentionally; denominator not defined; missing-period/time-boundary error; validation only repeats the same logic; DISTINCT used as unexplained repair; EXPLAIN optimization done before correctness.

FAST DEBUG LOOP

Run smallest stage -> count rows -> count distinct business key -> check null/orphan/duplicate behavior -> reconcile one control total -> repair one cause -> rerun controls. Never change five things at once.

COURSE POSITIONING

This C2A SQL depth is optional. C2B owns the fast-track engineering transfer. Strong C2A SQL may reduce repair needed in C2B, but C2A completion is not a prerequisite for C2B/C3.

C2-02 source / video registry

PostgreSQL 18 is authoritative; video is visual support only

ASQL1 - PostgreSQL current joins tutorial

https://www.postgresql.org/docs/current/tutorial-join.html

Alex 51:10-1:08:03

ASQL2 - PostgreSQL SELECT / sub-SELECT reference

https://www.postgresql.org/docs/current/sql-select.html

Book-led

ASQL3 - PostgreSQL WITH queries

https://www.postgresql.org/docs/current/queries-with.html

Alex 1:58:55-2:09:06

ASQL4 - PostgreSQL window tutorial

https://www.postgresql.org/docs/current/tutorial-window.html

Alex 1:45:58-1:58:54

ASQL5/6 - PostgreSQL window functions reference

https://www.postgresql.org/docs/current/functions-window.html

ASQL6 may use Alex window chapter visually

ASQL7 - PostgreSQL aggregate functions

https://www.postgresql.org/docs/current/functions-aggregate.html

Alex 1:26:39-1:35:15 CASE

ASQL8 - PostgreSQL 18 date/time functions

https://www.postgresql.org/docs/18/functions-datetime.html

Book-led

ASQL9 - PostgreSQL table/query expressions

https://www.postgresql.org/docs/current/queries-table-expressions.html

Book-led

ASQL10 - PostgreSQL SELECT reference

https://www.postgresql.org/docs/current/sql-select.html

Book-led

ASQL11 - PostgreSQL EXPLAIN + Indexes

https://www.postgresql.org/docs/current/using-explain.html

Book-led; current Indexes chapter also referenced

ASQL12 - PostgreSQL query chapter

https://www.postgresql.org/docs/current/queries.html

Alex 2:42:46-3:32:13 optional workflow visual

VERIFICATION NOTE

PostgreSQL 18/current documentation and the Alex video title/timestamps were rechecked on the web 2026-09-10. PostgreSQL syntax wins whenever the MySQL-oriented video differs. No external source is required to pass.